

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Шапошник А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 22.

Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2.
Дочерние процессы инвертируют строки.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; — создает дочерний процесс.
- `int pipe(int *fd)`; — создаёт однонаправленный канал (pipe).
- `int dup2(int oldfd, int newfd)`; — дублирует дескриптор `oldfd` в `newfd`, закрывая `newfd`.
- `int execv(const char *path, char *const argv[])`; — заменяет образ текущего процесса новой программой, загруженной из `path`.
- `ssize_t read(int fd, void *buf, size_t count)`; — читает до `count` байт из дескриптора `fd`.
- `ssize_t write(int fd, const void *buf, size_t count)`; — пишет до `count` байт в дескриптор `fd`.
- `int close(int fd)`; — закрывает файловый дескриптор.
- `pid_t wait(int *status)`; — ожидает завершения любого дочернего процесса.
- `int open(const char *path, int flags, mode_t mode)`; — открывает/создаёт файл, возвращает файловый дескриптор.
- `pid_t getpid(void)`; — возвращает PID текущего процесса.
- `time_t time(time_t *t)`; — возвращает текущее время.

Программа состоит из двух независимых исполняемых файлов:

1) `client` — родительский процесс. Читает строки со `stdin`, случайно (80/20) выбирает одного из двух детей, отправляет строку ему через pipe, читает ответ и пишет результат на экран и (опционально) в файл.

2) `reverse` — дочерний процесс-фильтр. Читает строки из `stdin` (перенаправленного на pipe), инвертирует каждую строку, пишет результат в `stdout`.

Алгоритм работы

1. Создание 4 pipe: `to_c1`, `to_c2` (клиент → дети), `from_c1`, `from_c2` (дети → клиент). Два pipe на каждого ребёнка, потому что Unix pipe однонаправленный.
2. Два `fork()`: создаются два дочерних процесса от родителя.
3. В каждом ребёнке:
 - Закрываются все ненужные концы pipe.
 - `dup2` перенаправляет `stdin` на входной pipe, `stdout` — на выходной.
 - `execv` заменяет образ процесса на программу `reverse`.
4. В родителе:
 - Закрываются ненужные концы pipe.
 - Открывается файл вывода (если передан).
 - Считывается строка, выбирается случайный ребенок (80/20), записывается строка и выводится ответ
5. Завершение: родитель закрывает пишущие концы pipe, дети получают EOF, завершаются; родитель вызывает `wait(NULL)` дважды.

Код программы

reverse.c

```
#include <unistd.h>
#include <stdlib.h>

#define BUF_SIZE 4096

static void reverse(char *s, ssize_t len) {
    for (ssize_t i = 0, j = len - 1; i < j; ++i, --j) {
        char tmp = s[i];
        s[i] = s[j];
        s[j] = tmp;
    }
}

static int write_all(int fd, const char *buf, ssize_t len) {
    ssize_t written = 0;
    while (written < len) {
        ssize_t w = write(fd, buf + written, len - written);
        if (w < 0) return -1;
        written += w;
    }
    return 0;
}

int main(void) {
    char buf[BUF_SIZE];
    ssize_t bytes;
    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        ssize_t start = 0;
        for (ssize_t i = 0; i < bytes; ++i) {
            if (buf[i] == '\n') {
                reverse(buf + start, i - start);
                start = i + 1;
            }
        }
        if (start < bytes) {
            reverse(buf + start, bytes - start);
        }

        if (write_all(STDOUT_FILENO, buf, bytes) == -1) {
            const char msg[] = "reverse-filter: write error\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
    }

    if (bytes < 0) {
        const char msg[] = "reverse-filter: read error\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

client.c

```
#include <stdbool.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <fcntl.h>
#include <time.h>

#define BUF_SIZE 4096
#define FILTER_NAME "reverse"

static int write_all(int fd, const char *buf, ssize_t len) {
    ssize_t written = 0;
    while (written < len) {
        ssize_t w = write(fd, buf + written, len - written);
        if (w < 0) return -1;
        written += w;
    }
    return 0;
}

static ssize_t read_line(int fd, char *buf, size_t max) {
    ssize_t total = 0;
    while (total < (ssize_t)max) {
        ssize_t r = read(fd, buf + total, 1);
        if (r < 0) return -1;
        if (r == 0) return total;
        ++total;
        if (buf[total - 1] == '\n') break;
    }
    return total;
}

static bool roll_80(void) {
    int fd = open("/dev/urandom", O_RDONLY);
    if (fd < 0) {
        unsigned seed = (unsigned)time(NULL) ^ (unsigned)getpid();
        return (seed % 100) < 80;
    }
    unsigned char byte;
    do {
        if (read(fd, &byte, 1) != 1) {
            close(fd);
            return true;
        }
    } while (byte >= 200);
    close(fd);
    return (byte % 100) < 80;
}

int main(int argc, char **argv) {
    if (argc < 2) {
        const char msg[] = "usage: client path/to/reverse-filter [output_file]\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_SUCCESS);
    }
}
```

```

const char *filterpath = argv[1];

int out_fd = STDOUT_FILENO;
if (argc >= 3) {
    out_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (out_fd == -1) {
        const char msg[] = "error: failed to open output file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
}

int to_c1[2], to_c2[2], from_c1[2], from_c2[2];
if (pipe(to_c1) == -1 || pipe(to_c2) == -1 ||
    pipe(from_c1) == -1 || pipe(from_c2) == -1) {
    const char msg[] = "error: failed to create pipe\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

pid_t c1 = fork();
if (c1 == -1) {
    const char msg[] = "error: fork failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
if (c1 == 0) {
    close(to_c1[1]);
    close(to_c2[0]); close(to_c2[1]);
    close(from_c1[0]);
    close(from_c2[0]); close(from_c2[1]);
    if (out_fd != STDOUT_FILENO) close(out_fd);

    dup2(to_c1[0], STDIN_FILENO);
    dup2(from_c1[1], STDOUT_FILENO);
    close(to_c1[0]);
    close(from_c1[1]);

    char *const args[] = { (char *)FILTER_NAME, NULL };
    execv(filterpath, args);

    const char msg[] = "error: exec failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

pid_t c2 = fork();
if (c2 == -1) {
    const char msg[] = "error: fork failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
if (c2 == 0) {
    close(to_c1[0]); close(to_c1[1]);
    close(to_c2[1]);
    close(from_c1[0]); close(from_c1[1]);

```

```

close(from_c2[0]);
if (out_fd != STDOUT_FILENO) close(out_fd);

dup2(to_c2[0], STDIN_FILENO);
dup2(from_c2[1], STDOUT_FILENO);
close(to_c2[0]);
close(from_c2[1]);

char *const args[] = { (char *)FILTER_NAME, NULL };
execv(filterpath, args);

const char msg[] = "error: exec failed\n";
write(STDERR_FILENO, msg, sizeof(msg) - 1);
exit(EXIT_FAILURE);
}

close(to_c1[0]);
close(to_c2[0]);
close(from_c1[1]);
close(from_c2[1]);

char line[BUF_SIZE];
char resp[BUF_SIZE];

while (true) {
    ssize_t n = read_line(STDIN_FILENO, line, sizeof(line) - 1);
    if (n < 0) {
        const char msg[] = "error: read from stdin failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    if (n == 0) break;
    if (n == 1 && line[0] == '\n') break;

    bool to_first = roll_80();
    int wfd = to_first ? to_c1[1] : to_c2[1];
    int rfd = to_first ? from_c1[0] : from_c2[0];

    if (write_all(wfd, line, n) == -1) {
        const char msg[] = "error: write to child failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    ssize_t total = 0;
    while (total < n) {
        ssize_t r = read(rfd, resp + total, n - total);
        if (r < 0) {
            const char msg[] = "error: read from child failed\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
        if (r == 0) break;
        total += r;
    }

    write_all(STDOUT_FILENO, resp, total);
}

```

```

    if (out_fd != STDOUT_FILENO) write_all(out_fd, resp, total);
}

close(to_c1[1]);
close(to_c2[1]);
close(from_c1[0]);
close(from_c2[0]);

wait(NULL);
wait(NULL);

if (out_fd != STDOUT_FILENO) close(out_fd);

return 0;
}

```

Протокол работы программы

Входной файл in.txt:

```

Test
tseT
fgffjlfjgl
laboratory task 1
operation system

```

Вывод:

```

./client ./reverse out.txt < in.txt
tseT
Test
lgjfljffgf
1 ksat yrotarobal
metsys noitarepo

```

Вывод

В ходе лабораторной работы была реализована клиент-серверная архитектура на основе анонимных каналов (pipe) и системных вызовов fork/execv. Родительский процесс маршрутизирует строки между двумя дочерними процессами с вероятностью 80/20, а дети инвертируют строки и возвращают результат обратно. Всего использовано 4 pipe (по 2 на каждого ребёнка), что обусловлено однонаправленностью Unix-каналов.